

Proyecto 7

alke_wallet_orm/settings.py

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'gestion',
]
# DATABASES = {
#     'default': {
#         'ENGINE': 'django.db.backends.postgresql',
#         'NAME': os.environ.get('DB_NAME', 'alke_wallet'),
#         'USER': os.environ.get('DB_USER', 'alke_user'),
#         'PASSWORD': os.environ.get('DB_PASSWORD', ''),
#         'HOST': os.environ.get('DB_HOST', 'localhost'),
#         'PORT': os.environ.get('DB_PORT', '5432'),
#     }
# }
LOGIN_URL = 'login'
LOGIN_REDIRECT_URL = 'cliente_list'
LOGOUT_REDIRECT_URL = 'login'
LANGUAGE_CODE = 'es'
```

gestion/models.py

```
from django.db import models
from django.urls import reverse

class Cliente(models.Model):
    nombre = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    telefono = models.CharField(max_length=20, blank=True, null=True)
    fecha_registro = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['nombre']

    def __str__(self):
        return self.nombre

class Cuenta(models.Model):
    """Relacion Uno a Uno: cada cliente tiene exactamente una cuenta."""
    cliente = models.OneToOneField(
        Cliente, on_delete=models.CASCADE, related_name='cuenta'
    )
    numero_cuenta = models.CharField(max_length=20, unique=True)
    saldo = models.DecimalField(max_digits=12, decimal_places=2, default=0)
    fecha_apertura = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f'Cuenta {self.numero_cuenta} de {self.cliente.nombre}'

class Etiqueta(models.Model):
    """Relacion Muchos a Muchos con Transaccion."""
```

```

nombre = models.CharField(max_length=40, unique=True)
def __str__(self):
    return self.nombre

class Transaccion(models.Model):
    """Relacion Muchos a Uno con Cuenta (una cuenta -> muchas transacciones)."""
    DEPOSITO = 'deposito'
    RETIRO = 'retiro'
    TRANSFERENCIA = 'transferencia'
    TIPO_CHOICES = [
        (DEPOSITO, 'Depósito'),
        (RETIRO, 'Retiro'),
        (TRANSFERENCIA, 'Transferencia'),
    ]

    cuenta = models.ForeignKey(
        Cuenta, on_delete=models.CASCADE, related_name='transacciones'
    )
    cuenta_destino = models.ForeignKey(
        Cuenta, on_delete=models.SET_NULL, null=True, blank=True,
        related_name='transferencias_recibidas',
        help_text='Solo aplica para transacciones de tipo Transferencia.'
    )
    tipo = models.CharField(max_length=15, choices=TIPO_CHOICES)
    monto = models.DecimalField(max_digits=12, decimal_places=2)
    fecha = models.DateTimeField(auto_now_add=True)
    etiquetas = models.ManyToManyField(Etiqueta, blank=True, related_name='transacciones')

    def __str__(self):
        return f'{self.get_tipo_display()} de {self.monto} en {self.cuenta.numero_cuenta}'

```

```

$ python manage.py makemigrations gestion
Migrations for 'gestion':
  gestion/migrations/0001_initial.py
    + Create model Cliente
    + Create model Etiqueta
    + Create model Cuenta
    + Create model Transaccion

$ python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, gestion, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying gestion.0001_initial... OK
  Applying sessions.0001_initial... OK

```

```

>>> from gestion.models import Cliente
>>> Cliente.objects.create(nombre="Ana García", email="ana2@example.com", telefono="123456789")
<Cliente: Ana García>
>>> Cliente.objects.create(nombre="Luis Pérez", email="luis2@example.com")
<Cliente: Luis Pérez>

>>> todos_clientes = Cliente.objects.all()
>>> print(list(todos_clientes))
[<Cliente: Ana García>, <Cliente: Luis Pérez>]

>>> clientes_A = Cliente.objects.filter(nombre__istartswith='A')
>>> print(list(clientes_A))
[<Cliente: Ana García>]

>>> cliente = Cliente.objects.get(email="luis2@example.com")
>>> cliente.telefono = "987654321"
>>> cliente.save()
OK, telefono actualizado a 987654321

>>> Cliente.objects.get(email="ana2@example.com").delete()
(1, {'gestion.Cliente': 1})

# --- Consulta SQL personalizada con raw() ---
>>> consulta = "SELECT * FROM gestion_cliente WHERE telefono IS NOT NULL"
>>> clientes_con_telefono = Cliente.objects.raw(consulta)
>>> for cliente in clientes_con_telefono:
...     print(cliente.nombre, cliente.email)
Luis Pérez luis2@example.com

# --- Consulta directa con cursor ---
>>> from django.db import connection
>>> with connection.cursor() as cursor:
...     cursor.execute("SELECT COUNT(*) FROM gestion_cliente")
...     resultado = cursor.fetchone()
...     print("Total de clientes:", resultado[0])
Total de clientes: 1

```

gestion/views.py

```

class CuentaListView(LoginRequiredMixin, ListView):
    model = Cuenta
    template_name = 'gestion/cuenta_list.html'
    context_object_name = 'cuentas'

    def get_queryset(self):
        # filter + annotate: cantidad de transacciones y monto total por cuenta
        return Cuenta.objects.select_related('cliente').annotate(
            total_transacciones=Count('transacciones'),
            monto_total=Sum('transacciones__monto'),
        )

class TransaccionListView(LoginRequiredMixin, ListView):
    model = Transaccion
    template_name = 'gestion/transaccion_list.html'
    context_object_name = 'transacciones'
    paginate_by = 20

    def get_queryset(self):
        qs = Transaccion.objects.select_related('cuenta', 'cuenta_destino').prefetch_related('etiquetas')
        # exclude(): permite ocultar depósitos con ?ocultar_depositos=1
        if self.request.GET.get('ocultar_depositos'):

```

```
qs = qs.exclude(tipo=Transaccion.DEPOSITO)
return qs.filter(Q(monto__gt=0))
```

gestion/views.py

```
from django.contrib.auth.mixins import LoginRequiredMixin
from django.db.models import Count, Sum, Q
from django.urls import reverse_lazy
from django.views.generic import ListView, CreateView, UpdateView, DeleteView
from .forms import ClienteForm, CuentaForm, TransaccionForm
from .models import Cliente, Cuenta, Transaccion
```

```
class ClienteListView(LoginRequiredMixin, ListView):
    model = Cliente
    template_name = 'gestion/cliente_list.html'
    context_object_name = 'clientes'
```

```
class ClienteCreateView(LoginRequiredMixin, CreateView):
    model = Cliente
    form_class = ClienteForm
    template_name = 'gestion/cliente_form.html'
    success_url = reverse_lazy('cliente_list')
```

```
class ClienteUpdateView(LoginRequiredMixin, UpdateView):
    model = Cliente
    form_class = ClienteForm
    template_name = 'gestion/cliente_form.html'
    success_url = reverse_lazy('cliente_list')
```

```
class ClienteDeleteView(LoginRequiredMixin, DeleteView):
    model = Cliente
    template_name = 'gestion/cliente_confirm_delete.html'
    success_url = reverse_lazy('cliente_list')
```

```
class TransaccionCreateView(LoginRequiredMixin, CreateView):
    model = Transaccion
    form_class = TransaccionForm
    template_name = 'gestion/transaccion_form.html'
    success_url = reverse_lazy('transaccion_list')
```

```
def form_valid(self, form):
    respuesta = super().form_valid(form)
    t = self.object
    if t.tipo == Transaccion.DEPOSITO:
        t.cuenta.saldo += t.monto; t.cuenta.save()
    elif t.tipo == Transaccion.RETIRO:
        t.cuenta.saldo -= t.monto; t.cuenta.save()
    elif t.tipo == Transaccion.TRANSFERENCIA and t.cuenta_destino:
        t.cuenta.saldo -= t.monto; t.cuenta.save()
        t.cuenta_destino.saldo += t.monto; t.cuenta_destino.save()
    return respuesta
```

gestion/forms.py

```
from django import forms
from .models import Cliente, Cuenta, Transaccion
```

```

class ClienteForm(forms.ModelForm):
    class Meta:
        model = Cliente
        fields = ['nombre', 'email', 'telefono']

    def clean_telefono(self):
        telefono = self.cleaned_data.get('telefono')
        if telefono and not telefono.replace('+', '').replace(' ', '').isdigit():
            raise forms.ValidationError('El teléfono solo puede contener números, espacios y "+".')
        return telefono

class TransaccionForm(forms.ModelForm):
    class Meta:
        model = Transaccion
        fields = ['cuenta', 'cuenta_destino', 'tipo', 'monto', 'etiquetas']
        widgets = {'etiquetas': forms.CheckboxSelectMultiple}

    def clean(self):
        datos = super().clean()
        if datos.get('tipo') == Transaccion.TRANSFERENCIA and not datos.get('cuenta_destino'):
            raise forms.ValidationError('Las transferencias requieren una cuenta destino.')
        if datos.get('monto') is not None and datos['monto'] <= 0:
            raise forms.ValidationError('El monto debe ser mayor a cero.')
        return datos

```

gestion/urls.py

```

from django.urls import path
from . import views

urlpatterns = [
    path("", views.ClienteListView.as_view(), name='cliente_list'),
    path('clientes/nuevo/', views.ClienteCreateView.as_view(), name='cliente_create'),
    path('clientes/<int:pk>/editar/', views.ClienteUpdateView.as_view(), name='cliente_update'),
    path('clientes/<int:pk>/eliminar/', views.ClienteDeleteView.as_view(), name='cliente_delete'),

    path('cuentas/', views.CuentaListView.as_view(), name='cuenta_list'),
    path('cuentas/nueva/', views.CuentaCreateView.as_view(), name='cuenta_create'),
    path('cuentas/<int:pk>/editar/', views.CuentaUpdateView.as_view(), name='cuenta_update'),
    path('cuentas/<int:pk>/eliminar/', views.CuentaDeleteView.as_view(), name='cuenta_delete'),

    path('transacciones/', views.TransaccionListView.as_view(), name='transaccion_list'),
    path('transacciones/nueva/', views.TransaccionCreateView.as_view(), name='transaccion_create'),
    path('transacciones/<int:pk>/editar/', views.TransaccionUpdateView.as_view(),
name='transaccion_update'),
    path('transacciones/<int:pk>/eliminar/', views.TransaccionDeleteView.as_view(),
name='transaccion_delete'),
]

```

gestion/admin.py

```

from django.contrib import admin
from .models import Cliente, Cuenta, Etiqueta, Transaccion

class CuentaInline(admin.StackedInline):
    model = Cuenta

```

```

    extra = 0
@admin.register(Cliente)
class ClienteAdmin(admin.ModelAdmin):
    list_display = ('nombre', 'email', 'telefono', 'fecha_registro')
    search_fields = ('nombre', 'email')
    inlines = [CuentaInline]

class TransaccionInline(admin.TabularInline):
    model = Transaccion
    fk_name = 'cuenta'
    extra = 0
    fields = ('tipo', 'monto', 'fecha')
    readonly_fields = ('fecha',)

@admin.register(Cuenta)
class CuentaAdmin(admin.ModelAdmin):
    list_display = ('numero_cuenta', 'cliente', 'saldo', 'fecha_apertura')
    list_filter = ('fecha_apertura',)
    search_fields = ('numero_cuenta', 'cliente__nombre')
    inlines = [TransaccionInline]

@admin.register(Transaccion)
class TransaccionAdmin(admin.ModelAdmin):
    list_display = ('id', 'cuenta', 'tipo', 'monto', 'fecha', 'cuenta_destino')
    list_filter = ('tipo', 'fecha')
    search_fields = ('cuenta__numero_cuenta', 'cuenta__cliente__nombre')
    filter_horizontal = ('etiquetas',)

```

gestion/tests.py

```

class ModeloRelacionesTest(TestCase):
    def setUp(self):
        self.cliente = Cliente.objects.create(nombre='Ana García', email='ana@example.com')
        self.cuenta = Cuenta.objects.create(cliente=self.cliente, numero_cuenta='AW-0001', saldo=1000)

    def test_relacion_uno_a_uno_cliente_cuenta(self):
        self.assertEqual(self.cliente.cuenta, self.cuenta)

    def test_relacion_muchos_a_uno_transaccion_cuenta(self):
        Transaccion.objects.create(cuenta=self.cuenta, tipo=Transaccion.DEPOSITO, monto=200)
        Transaccion.objects.create(cuenta=self.cuenta, tipo=Transaccion.RETIRO, monto=50)
        self.assertEqual(self.cuenta.transacciones.count(), 2)

    def test_relacion_muchos_a_muchos_transaccion_etiqueta(self):
        etiqueta = Etiqueta.objects.create(nombre='servicios')
        t = Transaccion.objects.create(cuenta=self.cuenta, tipo=Transaccion.RETIRO, monto=80)
        t.etiquetas.add(etiqueta)
        self.assertIn(etiqueta, t.etiquetas.all())

class ConsultasPersonalizadasTest(TestCase):
    def test_filter_exclude(self): ...
    def test_annotate_conteo_por_cuenta(self): ...
    def test_raw_query(self):
        resultados = list(Cuenta.objects.raw(
            'SELECT * FROM gestion_cuenta WHERE saldo >= %s', [500]
        ))
        self.assertEqual(len(resultados), 1)

```

```
class VistasSeguridadTest(TestCase):
    def test_usuario_anonimo_redirigido_al_login(self): ...
    def test_crear_cliente_autenticado(self): ...
```

```
$ python manage.py test -v 2
Found 8 test(s).
Creating test database for alias 'default'...
Applying gestion.0001_initial... OK
System check identified no issues (0 silenced).
test_annotate_conteo_por_cuenta ... ok
test_filter_exclude ... ok
test_raw_query ... ok
test_relacion_muchos_a_muchos_transaccion_etiqueta ... ok
test_relacion_muchos_a_uno_transaccion_cuenta ... ok
test_relacion_uno_a_uno_cliente_cuenta ... ok
test_crear_cliente_autenticado ... ok
test_usuario_anonimo_redirigido_al_login ... ok

-----
Ran 8 tests in 1.439s

OK
Destroying test database for alias 'default'...
```

Alke Wallet

[Clientes](#)
[Cuentas](#)
[Transacciones](#)
[Iniciar sesión](#)

Iniciar sesión

Nombre de usuario:

Contraseña:

Ingresar

Alke Wallet

[Clientes](#)
[Cuentas](#)
[Transacciones](#)

Hola, **gestor**

Cerrar sesión

Clientes

+ Nuevo cliente

NOMBRE	EMAIL	TELEFONO	CUENTA	
Ana García	ana@alkewallet.com	+56 9 1234 5678	AW-1001	Editar Eliminar
Bruno Silva	bruno@alkewallet.com	+56 9 8765 4321	AW-1002	Editar Eliminar
Carla Díaz	carla@alkewallet.com	—	AW-1003	Editar Eliminar
Luis Pérez	luis2@example.com	987654321	sin cuenta	Editar Eliminar

Alke Wallet

Cientes

Cuentas

Transacciones

Hola, **gestor**

Cerrar sesión

Nuevo cliente

Nombre:

Email:

Telefono:

Guardar

Cancelar

Alke Wallet

Cientes

Cuentas

Transacciones

Hola, **gestor**

Cerrar sesión

Cuentas

+ Nueva cuenta

N° CUENTA	CLIENTE	SALDO	TRANSACCIONES	MONTO TOTAL MOVIDO	
AW-1001	Ana García	\$20000,00	2	\$70000	Editar Eliminar
AW-1002	Bruno Silva	\$80000,00	1	\$15000	Editar Eliminar
AW-1003	Carla Díaz	\$40000,00	1	\$10000	Editar Eliminar

Columnas "Transacciones" y "Monto total movido" calculadas con `awaitar(Cuentas).1.Serie...3`.

Alke Wallet

Cientes

Cuentas

Transacciones

Hola, **gestor**

Cerrar sesión

Transacciones

+ Nueva transacción

CUENTA	TIPO	MONTO	DESTINO	ETIQUETAS	FECHA	
AW-1003	Depósito	\$10000,00	—	ingreso	26/09/2026 21:49	Editar Eliminar
AW-1001	Transferencia	\$20000,00	AW-1003	compra	26/09/2026 21:49	Editar Eliminar
AW-1002	Retiro	\$15000,00	—	servicio	26/09/2026 21:49	Editar Eliminar
AW-1001	Depósito	\$50000,00	—	ingreso	26/09/2026 21:49	Editar Eliminar

Agregó `location_deposito=1` a la URL para ver el filtro con exclusión en acción.

Nueva transacción

Cuentas:
- Select an option -

Cuenta destino:
- Select an option -

Solo aplica para transacciones de tipo Transferencia.

Tipo:
- Select an option -

Monto:

Etiquetas:

compras

☐

ingreso

☐

servicios

☐

Guardar

Cancelar

Transacciones

[+ Nueva transacción](#)

CUENTA	TIPO	MONTO	DESTINO	ETIQUETAS	FECHA		
AW-1001	Transferencia	\$20000,00	AW-1003	compra	26/09/2026 21:49	Editar	Eliminar
AW-1002	Retiro	\$15000,00	—	servicio	26/09/2026 21:49	Editar	Eliminar

Agrega ?excluir=?deposits=1 a la URL para ver el filtro con exclude() en acción.

Sito administrativo

AUTENTICACIÓN Y AUTORIZACIÓN

Grupos	+ Añadir	✎ Modificar
Usuarios	+ Añadir	✎ Modificar

GESTIÓN

Cientes	+ Añadir	✎ Modificar
Cuentas	+ Añadir	✎ Modificar
Etiquetas	+ Añadir	✎ Modificar
Transacciones	+ Añadir	✎ Modificar

Acciones recientes

Mis acciones

Ninguna disponible

Inicio > Gestion > Clientes

Seleccione cliente a modificar

ANADIR CLIENTE +

Q

Buscar

Acción: - Select an option -

Run

seleccionados 0 de 4

☐	NOMBRE	EMAIL	TELEFONO	FECHA REGISTRO
☐	Ana García	ana@alkewallet.com	+56 9 1234 5678	26 de septiembre de 2026 a las 21:49
☐	Bruno Silva	bruno@alkewallet.com	+56 9 8765 4321	26 de septiembre de 2026 a las 21:49
☐	Carla Díaz	carla@alkewallet.com	-	26 de septiembre de 2026 a las 21:49
☐	Luis Pérez	luis2@example.com	987654321	26 de septiembre de 2026 a las 21:48

4 clientes

Inicio > Gestion > Clientes > Ana García

Modificar cliente

HISTORICO

VER EN EL SITIO >

Ana García

Nombre:

Ana García

Email:

ana@alkewallet.com

Telefono:

+56 9 1234 5678

CUENTA:

Cuenta: Cuenta AW-1001 de Ana García [Ver en el sitio](#)

☐ Eliminar

Numero cuenta:

AW-1001

Saldo:

250000.00

GUARDAR

Guardar y añadir otro

Guardar y continuar editando

Eliminar

Inicio > Gestion > Transacciones

Seleccione transaccion a modificar

ANADIR TRANSACCION +

Q

Buscar

Acción: - Select an option -

Run

seleccionados 0 de 4

☐	ID	CUENTA	TIPO	MONTO	FECHA	CUENTA DESTINO
☐	4	Cuenta AW-1003 de Carla Díaz	Depósito	10000.00	26 de septiembre de 2026 a las 21:49	-
☐	3	Cuenta AW-1001 de Ana García	Transferencia	20000.00	26 de septiembre de 2026 a las 21:49	Cuenta AW-1003 de Carla Díaz
☐	2	Cuenta AW-1002 de Bruno Silva	Retiro	15000.00	26 de septiembre de 2026 a las 21:49	-
☐	1	Cuenta AW-1001 de Ana García	Depósito	50000.00	26 de septiembre de 2026 a las 21:49	-

4 transacciones

README.md

Alke Wallet — Módulo 7 (Django ORM)

Aplicación Django que modela clientes, cuentas y transacciones de una billetera digital, con ORM, migraciones, consultas personalizadas y CRUD.

Instalación

```

```
python -m venv venv && source venv/bin/activate
pip install django
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

```

Modelos y relaciones

- Cliente 1—1 Cuenta (OneToOneField)
- Cuenta 1—N Transaccion (ForeignKey)
- Transaccion N—N Etiqueta (ManyToManyField)

Consultas personalizadas

- filter()/exclude() en TransaccionListView (?ocultar_depositos=1)
- annotate(Count, Sum) en CuentaListView
- raw() y cursor directo demostrados en la shell (ver sección 5 del informe)

Pruebas

```

```
python manage.py test
```

```

Producción (PostgreSQL)

Ver comentario en settings.py con la configuración de DATABASES y la instalación de psycopg2-binary.

11. Retos enfrentados durante el desarrollo

- Modelar las tres relaciones pedidas en un mismo dominio coherente: se resolvió usando Cliente↔Cuenta como 1:1 (cada cliente tiene una única billetera), Cuenta↔Transacción como 1:N (historial de movimientos) y Transacción↔Etiqueta como N:M (categorización flexible sin modificar el modelo de transacción).
- Transferencias entre cuentas: una transacción de tipo "Transferencia" necesita afectar el saldo de dos cuentas distintas (origen y destino). Se resolvió con una segunda ForeignKey (cuenta_destino, opcional) y lógica en form_valid() de TransaccionCreateView para actualizar ambos saldos de forma consistente.
- Consultas eficientes: al listar cuentas con su cantidad de transacciones y monto total, una implementación ingenua generaría una consulta N+1. Se resolvió combinando select_related('cliente') con annotate(Count(...), Sum(...)) para resolver todo en una sola consulta SQL.
- Demostrar raw() y cursores sin duplicar lógica: en vez de simularlos, se ejecutó una sesión real en la shell de Django (python manage.py shell) contra la base SQLite del proyecto, capturando la salida real de cada comando, incluyendo el conteo por cursor directo a gestion_cliente.
- Renderizado de capturas del admin: el motor de renderizado usado (basado en WebKit) no calculaba bien la altura de páginas con CSS grid; se resolvió fijando una altura explícita al generar cada captura del panel de administración.

<https://github.com/amatamalaastudillo-dot/Proyecto7.git>